Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 — Midterm Project

Team leader: TEAM_LEADER · **Members:** TEAM_MEMBERS

Code: github.com/samridhivasundhara-sv/CMPSC448-Midterm-Project

Summary. We built a balanced dataset of 20,000 real (prompt, response) pairs from Chatbot Arena, covering five LLM families — Claude, GPT, Google (Gemini / PaLM-2), Llama-2 and Mistral — and trained a convolutional network (CNN) and a bidirectional LSTM to predict which family wrote a response. **RQ1:** from the response alone, the BiLSTM reaches 66.7% accuracy and the CNN 64.7% (chance = 20%). **RQ2:** the prompt alone is useless (≈21%, chance level) because in Arena every prompt is answered by two different models; adding the prompt to the response slightly *hurts*. **RQ3:** fingerprints transfer to unseen task types with a 6–9 point F1 drop; Claude is identifiable almost everywhere, Google is the most task-dependent. **RQ4:** the strongest signals are structural (markdown, bullets, bold text, an introductory "Here is …:" line, paragraphing) together with characteristic phrases ("Sure!", "Note:", "Remember,"); 29 hand-crafted style numbers alone reach 0.59 macro-F1, and masking all content words does not reduce accuracy at all.

1. Problem definition

Given a piece of text produced by a large language model, can we tell which model produced it? We frame this as **supervised multi-class text classification**. Each example is a triple `(LLM_name, LLM_input, LLM_output)`; the label is the **model family** (5 classes: Claude, GPT, Google, Llama, Mistral). Depending on the research question the classifier sees the response (`LLM_output`), the prompt (`LLM_input`) or both. We predict the *family* rather than the exact version (e.g. `gpt-4-0613`) because versions of the same family share training recipes and style, and because the per-version counts are small and very unbalanced.

We report **accuracy** and **macro-F1** (the unweighted mean of the per-class F1 scores). Classes are balanced, so the two are nearly identical; chance level is 0.20 for both. Neural results are the mean ± standard deviation over **3 random seeds**.

2. Dataset curation

2.1 Source

We use `lmarena-ai/arena-human-preference-55k`, a dataset publicly released on Hugging Face by the LMSYS / LMArena team (Chatbot Arena). On the Arena website a user types a prompt, two *anonymous* models answer it side by side, and the user votes for the better answer. Each of the 57,477 rows contains the prompt, the two model names and the two responses. This source suits the project well: (i) the responses are real chatbot outputs, not text we wrote; (ii) the prompts are real user prompts across many topics; (iii) it covers more than 60 models from all the major families; and (iv) because the *same prompt* is answered by two different models, it is a controlled setting for RQ2. The data is public and was released for research; we use only the prompt/response text and model names (no user identifiers are included in the dataset).

2.2 Construction (`src/prepare_data.py`)

1. **First turn only.** Prompts and responses are stored as JSON lists of turns (≈13% of rows are multi-turn). We keep the first prompt and the first response so every example is a single (input, output) pair.
2. **Model → family.** We map versions by name prefix: `gpt-*` → GPT, `claude-*` → Claude, `llama-2-*` → Llama (Code Llama is excluded), `mistral-*` / `mixtral-*` → Mistral, `gemini-*` / `palm-*` → Google. All other models (Vicuna, Yi, Qwen, …) are discarded.
3. **Cleaning.** We drop null or empty responses and responses with fewer than 3 words.
4. **Task label (for RQ3).** Each prompt receives a coarse task type from an ordered keyword rule: *code* (programming words, code fences), then *math* (solve, calculate, equations, arithmetic), then *writing* (write, poem, story, email, essay, …), otherwise *qa*. The rule is simple and imperfect, but it is transparent and the categories it produces are clearly different (see RQ3).
5. **Balancing.** Before balancing we had 31.0k GPT, 15.8k Claude, 8.6k Mistral, 7.8k Llama and 4.6k Google responses. We randomly sample **4,000 responses per family** (20,000 total).
6. **Leak-free split.** Arena contains many repeated prompts (e.g. "test" appears 39 times, one survey prompt 101 times). We therefore split **by normalised prompt text** — 70% / 10% / 20% of the *distinct prompts* go to train / validation / test — so the same prompt never appears in two splits. This gives 13,922 training, 2,108 validation and 3,970 test examples.

Family	Model versions included (count)	Total
Claude	claude-2.1 (1388), claude-instant-1 (1022), claude-1 (992), claude-2.0 (598)	4000
GPT	gpt-3.5-turbo-0613/1106/0314/0125 (1654), gpt-4-1106-preview (909), gpt-4-0613 (796), gpt-4-0314 (498), gpt-4-0125-preview (143)	4000
Google	palm-2 (1714), gemini-pro-dev-api (1195), gemini-pro (1091)	4000
Llama	llama-2-70b-chat (1776), llama-2-13b-chat (1317), llama-2-7b-chat (907)	4000
Mistral	mixtral-8x7b-instruct (1686), mistral-medium (1537), mistral-7b-instruct (+v0.2) (777)	4000

Table 1. Composition of the dataset (`data/dataset.csv`).



Figure 1. Task types per family. Task mix is nearly identical across families (QA ≈ 67%, code ≈ 13%, writing ≈ 13%, math ≈ 7%), because every Arena prompt is shown to random pairs of models.

Things to keep in mind about the data. (a) A family label mixes several versions and sizes (e.g. GPT-3.5 and GPT-4), which makes the task harder and more realistic. (b) All responses date from 2023 – early 2024; newer models are not included. (c) Explicit self-identification is almost absent: fewer than 0.5% of responses in any family mention "OpenAI", "Anthropic", "Claude", "GPT", "Google", "Llama" or "Mistral" (we measured this), most likely because Arena filters out battles where a model reveals its name. So the classifiers cannot simply look for "As a model trained by OpenAI".

3. Models and training

3.1 Text processing (`src/common.py`)

We use a simple regular-expression tokenizer, `\n | \w+ | [^\w\s]`, that keeps **case, punctuation and line breaks** (a newline becomes the token `<nl>`). This is deliberate: markdown symbols (`**`, `#`, `-`, `1.`) and layout are part of a model's style, and a typical NLP pre-processing step (lower-casing, removing punctuation) would throw them away (RQ4 measures exactly how much that costs). The vocabulary is the 30,000 most frequent training tokens with frequency ≥ 2; anything else maps to `<unk>`. Sequences are truncated to 384 response tokens (the median response is ≈150 words) and 128 prompt tokens. For the "input + output" view the model reads `prompt[:128] <sep> response[:384]`.

3.2 CNN

A Kim-style (2014) text CNN. Token ids → 128-dimensional embeddings (trained from scratch, dropout 0.2) → three parallel 1-D convolutions with kernel widths 3, 4 and 5 (128 filters each, ReLU) → global max-pooling over time (padding positions are masked out) → concatenation (384 features) → dropout 0.5 → linear layer with 5 outputs. Each filter is effectively a learned detector for a 3-, 4- or 5-token pattern such as "`Sure ! Here`" or "`<nl> - **`", and max-pooling asks "does this pattern occur anywhere in the response?". Parameters: ≈4.0M (3.8M of them in the embedding table).

3.3 RNN (bidirectional LSTM)

Embeddings (128-d) → **2-layer bidirectional LSTM** with 128 hidden units per direction (dropout 0.3 between layers) → for every position the forward and backward states are concatenated (256-d) → the sequence is summarised by **masked mean-pooling and max-pooling** (512-d) → dropout → linear layer. Pooling over all time steps worked better in early tests than only using the last hidden state, because the evidence (e.g. a closing "Let me know if…") can be anywhere in a long response.

Implementation note. PyTorch's standard way of handling variable-length sequences (`pack_padded_sequence`) was ≈8× slower on the Apple-silicon GPU we used (222 s per epoch). We therefore wrote the bidirectional LSTM ourselves: the backward LSTM reads each sequence reversed *within its true length* (padding stays at the end), and its outputs are reversed back. We checked numerically that this gives the same outputs as PyTorch's packed bidirectional LSTM (max difference $4 \cdot 10^{-8}$). Combined with length-bucketed batches, an epoch takes ≈27 s.

3.4 Training procedure

- Loss: cross-entropy. Optimiser: AdamW (learning rate $1 \cdot 10^{-3}$, weight decay 10^{-4}), batch size 64, gradient-norm clipping at 1.0.

- Up to 15 epochs with **early stopping** on validation macro-F1 (patience 4); the weights from the best validation epoch are restored before testing. The test set is used only once per trained model.
- Batches group sequences of similar length (less padding); padded length is rounded up to a multiple of 32.
- Every configuration is trained with seeds 0, 1, 2. Hardware: Apple M2 Pro (PyTorch MPS backend). One CNN run takes ≈1.5 min, one BiLSTM run ≈6 min.
- **Hyper-parameter tuning** used the validation set only (`results/tuning.txt`). The first CNN applied dropout 0.5 to the embeddings and stopped after 10 epochs; it learned slowly and erratically (validation F1 0.55). Reducing embedding dropout to 0.2 and allowing 15 epochs raised validation F1 to 0.63; a larger learning rate ($2 \cdot 10^{-3}$) was worse (0.58). The BiLSTM reached its best validation F1 (0.67) around epoch 9–13 while training loss kept falling, which is the over-fitting point that early stopping is there to catch (Figure 4).

3.5 Reference model

To judge whether the neural models are good, we also train a non-neural baseline: TF-IDF over word unigrams and bigrams (case and punctuation kept, 100k features, sub-linear tf) + multinomial logistic regression (C = 5). It is deterministic, so it has no standard deviation.

4. Results

4.1 RQ1 — Can we identify which LLM generated a response?

Yes, about two times out of three. Using only the response text:

Model (output only)	Accuracy	Macro-F1	Best epoch (3 seeds)
Chance	0.200	0.200	–
CNN	0.647 ± 0.006	0.645 ± 0.005	14, 11, 9*
BiLSTM	**0.667 ± 0.007**	**0.666 ± 0.009**	7, 10, 9
TF-IDF + LogReg (reference)	0.670	0.667	–

Table 2. Test results for RQ1 (3,970 test responses, mean ± std over 3 seeds). *One CNN seed peaked at epoch 14 of 15, so a longer schedule might add a little for the CNN.

The BiLSTM is 2 points better than the CNN, and the difference is larger than the seed-to-seed spread. The LSTM reads the whole sequence and can combine evidence from the opening, the layout and the closing of a response, while the CNN only reports whether short local patterns occur. Interestingly, the simple TF-IDF model is as good as the BiLSTM: with 14k training examples and embeddings learned from scratch, the neural models have no advantage from pre-trained language knowledge, and much of the signal is in specific word pairs (see RQ4).

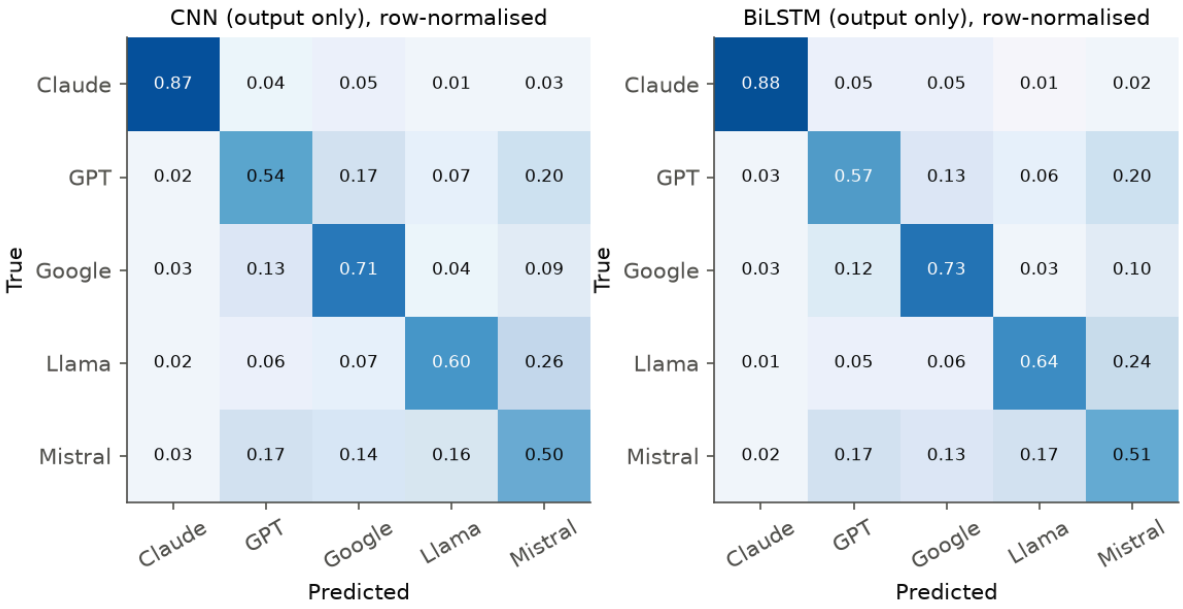


Figure 2. Row-normalised confusion matrices (sum over 3 seeds). Diagonal = recall per family.

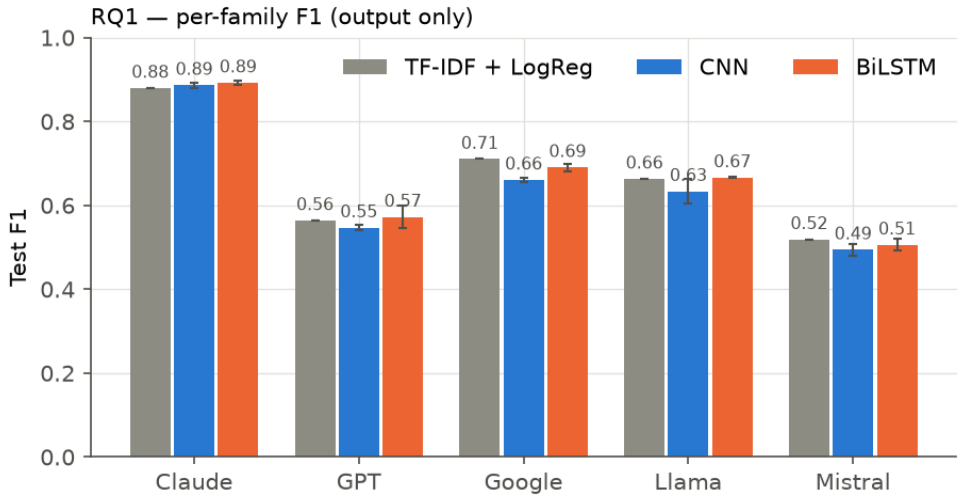


Figure 3. Per-family F1. Claude is by far the easiest family (F1 ≈ 0.89); Mistral is the hardest (≈ 0.50).

Which families get confused? Claude is recognised 87–88% of the time and rarely mistaken for anything else. The main confusions are **Llama → Mistral** (24–26% of Llama responses) and **GPT ↔ Mistral** (≈20%). Mistral's style sits "between" the others: its average length, list use and lexical diversity are all close to the overall average (RQ4), so it has few unique cues and acts as a catch-all class, which explains its low F1. One possible reason is that open instruction-tuned models are often trained on data written by other chat models, but our data cannot confirm this.

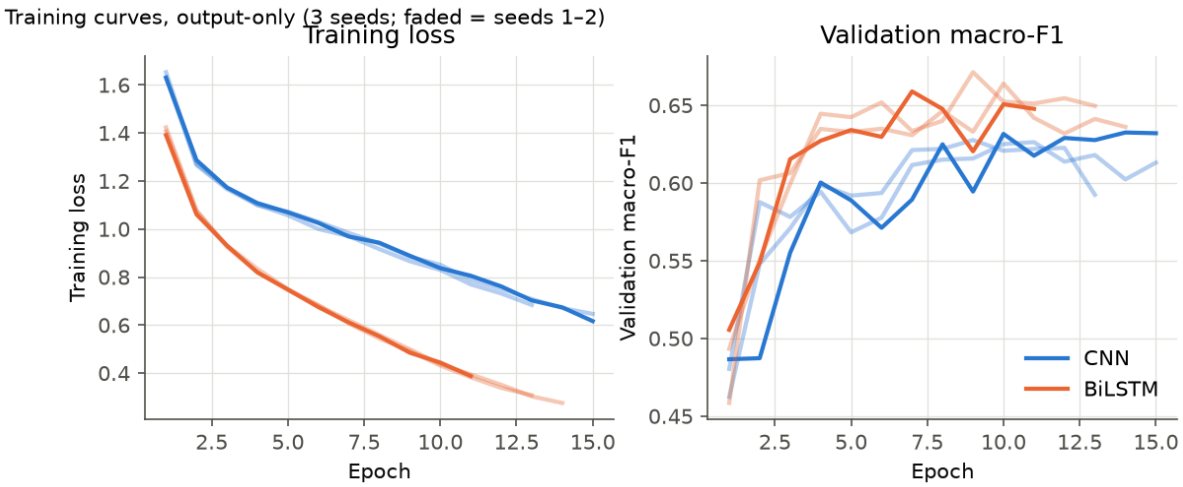


Figure 4. Training loss and validation macro-F1 per epoch (output-only). The BiLSTM fits the training data much faster; its validation score flattens after ≈7–9 epochs while the loss keeps dropping, so early stopping matters.

4.2 RQ2 — Does the user's prompt help identify the LLM?

Input given to the classifier	TF-IDF + LogReg	CNN	BiLSTM
Input only (prompt)	0.218	0.200 ± 0.014	0.214 ± 0.008
Output only (response)	**0.667**	**0.645 ± 0.005**	**0.666 ± 0.009**
Input + Output	0.646	0.630 ± 0.015	0.641 ± 0.005

Table 3. Test macro-F1 for the three input views (chance = 0.20).

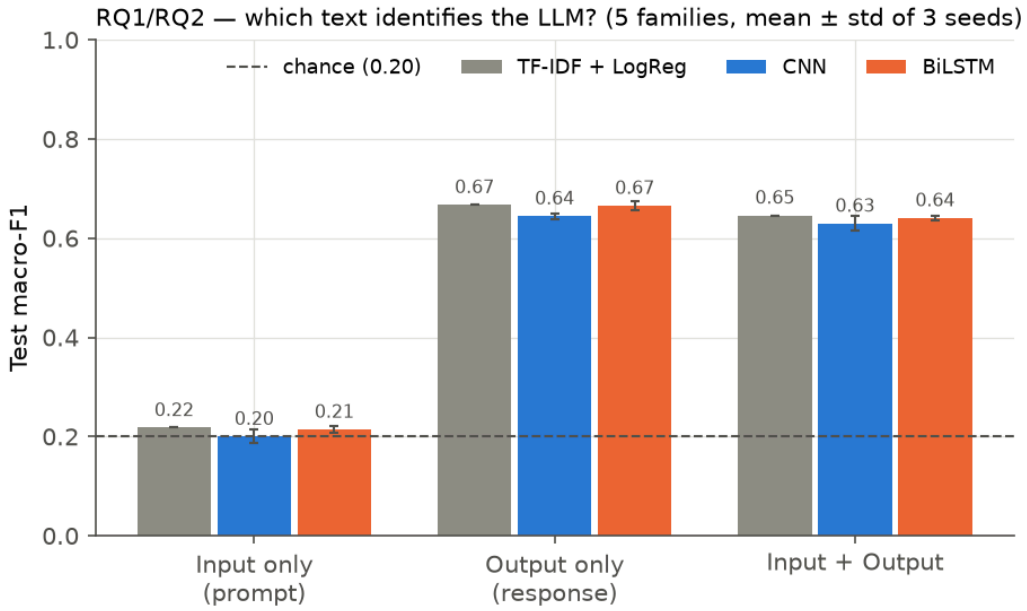


Figure 5. RQ1/RQ2 summary: only the response carries information about the author.

(i) Can the LLM be predicted from the prompt alone? No. All three classifiers are at chance level (0.20–0.22). This is what the data-collection process predicts. In Chatbot Arena, models are assigned to a prompt at random, and each prompt is answered by *two* models. For the 720 test rows (18%) whose battle partner

is also in the test set, a prompt-only classifier necessarily outputs the same label for both and can be right at most half the time; it is actually right 20.7% of the time. The prompt population is also the same for every family: median prompt length is 14–15 words for all five, and the task mix differs by at most 2.5 points (Figure 1). The "most predictive prompt words" the TF-IDF model learns are noise words (*goats, pokemon, terraform, Ashley*) — memorised training prompts that do not generalise.

(ii) Does including the input improve classification? No — it slightly hurts (-1.5 to -2.5 F1 points for all three models). The prompt adds tokens that contain no information about the author, so the models have more chances to fit noise, and (for the neural models) the prompt uses up part of the input length.

(iii) When could input-only work "surprisingly well"? Only when the prompt and the label are linked by how the data was collected, not by the models. Examples: each model was queried with its own set of prompts (e.g. GPT on coding datasets, Claude on creative writing); models were used during different time periods so prompts mention different events; or one model was only used in one language or one application. Then a prompt-only classifier learns "who was asked this kind of question", which is a **data-collection leak**, not an LLM fingerprint. Our result (chance level) is evidence that our dataset is free from this leak, so the output-only scores in RQ1 reflect the models' actual writing.

5. In-depth analyses

5.1 RQ3 — Do LLM fingerprints generalise across tasks?

We train on some task types and test on a task type the model never saw: **Setting A**: train on code + math + QA, test on **writing**; **Setting B**: train on general QA only, test on **code**. For each setting we compare three numbers: in-domain test F1 (unseen prompts of the training task types), the score on the target task of the RQ1 model that *did* see that task type during training, and the cross-task score.

	A: → writing			B: QA → code		
Model	**in-domain**	**seen**	**unseen**	**in-domain**	**seen**	**unseen**
TF-IDF + LogReg	0.658	0.664	0.601	0.667	0.603	0.536
CNN	0.645 ± .002	0.609	0.559 ± .013	0.648 ± .004	0.651	0.564 ± .018
BiLSTM	0.659 ± .003	0.620	**0.596 ± .011**	0.652 ± .007	0.683	**0.580 ± .007**

Table 4. Macro-F1. "seen" = model trained on all task types (RQ1), evaluated on the same target test set. Training sizes: A 12,155; B 9,339; RQ1 13,922.

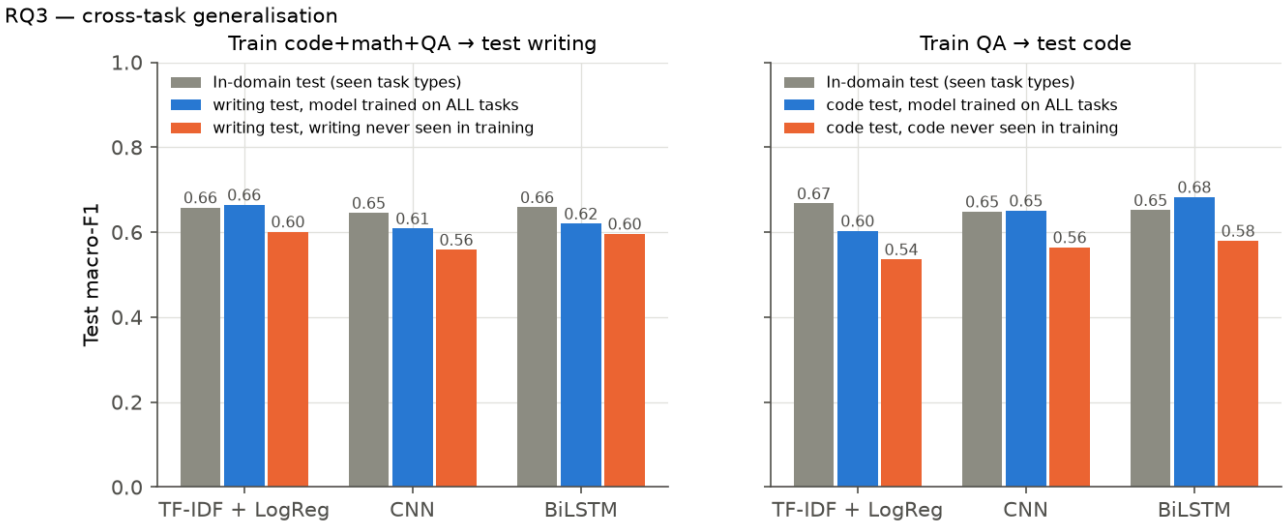


Figure 6. Cross-task generalisation. Orange bars (target task never seen) are lower than the grey and blue bars in every case.

(i) Does performance drop on unseen tasks? Yes, but moderately. F1 drops by 6–9 points relative to in-domain and by 2–10 points relative to a model that saw the task type. All scores stay at ≈3× chance, so most of the fingerprint transfers. The drop is larger for **QA → code** than for → **writing**: code answers are dominated by code blocks whose content is dictated by the problem, so the style cues learned from prose are partly missing, while writing tasks still contain the model's usual prose. The BiLSTM transfers best in both settings; TF-IDF suffers most on code (its features are specific word pairs, which change completely between topics).

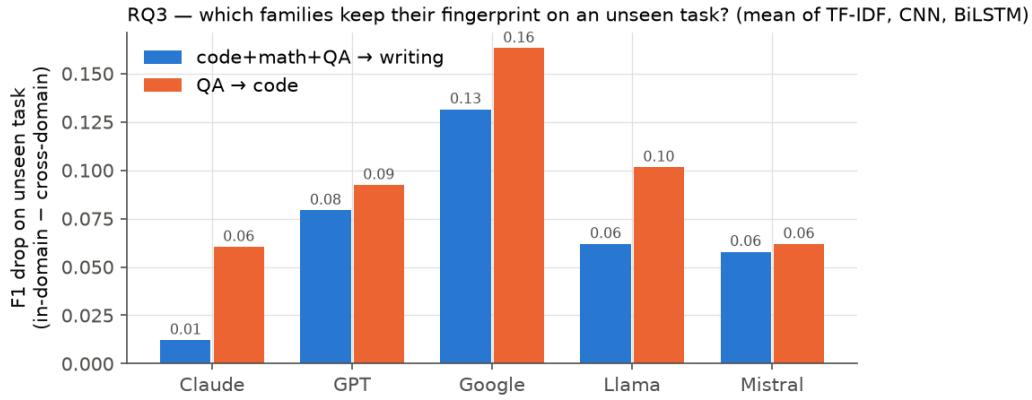


Figure 7. Per-family F1 drop when the target task is unseen (average of the three models).

(ii) Are some models more consistently identifiable? Yes. Claude barely changes (drop 0.01 on writing, 0.06 on code; cross-task F1 0.76–0.88) — its habits (an introduction line ending in a colon, bullet lists, "In summary", closing offers) show up whatever the task. **Google** is the least consistent (drop 0.13 and 0.16): its most typical cue, heavy **bold** markdown, depends on the type of answer (structured explanations) and partly disappears in poems, stories and code.

(iii) General fingerprint or task-specific patterns? Both. A general fingerprint clearly exists (60% accuracy on a never-seen task type). But a part of what the classifiers learn is task-specific: how a family formats a *certain kind* of answer. A classifier meant to be used in practice should therefore be trained on a mix of tasks.

5.2 RQ4 — What characteristics distinguish different LLMs?

We answer "why" with four complementary analyses (`src/rq4.py`).

(a) Measurable style differences. We computed 29 interpretable features per response in five groups: *length* (words, sentences, average sentence/word length), *lexical* (type–token ratio on the first 100 words, hapax ratio, rates of "I" and "you", upper-case and digit ratios), *formatting* (lines, paragraphs, bullet and numbered lines, markdown headers, bold spans, code blocks, first line ending in a colon), *punctuation* (rates of , ! ? : (and emoji) and *phrasing* (starts with "Sure/Certainly/Of course", apology or refusal, mentions being an AI, ends with an offer such as "let me know").

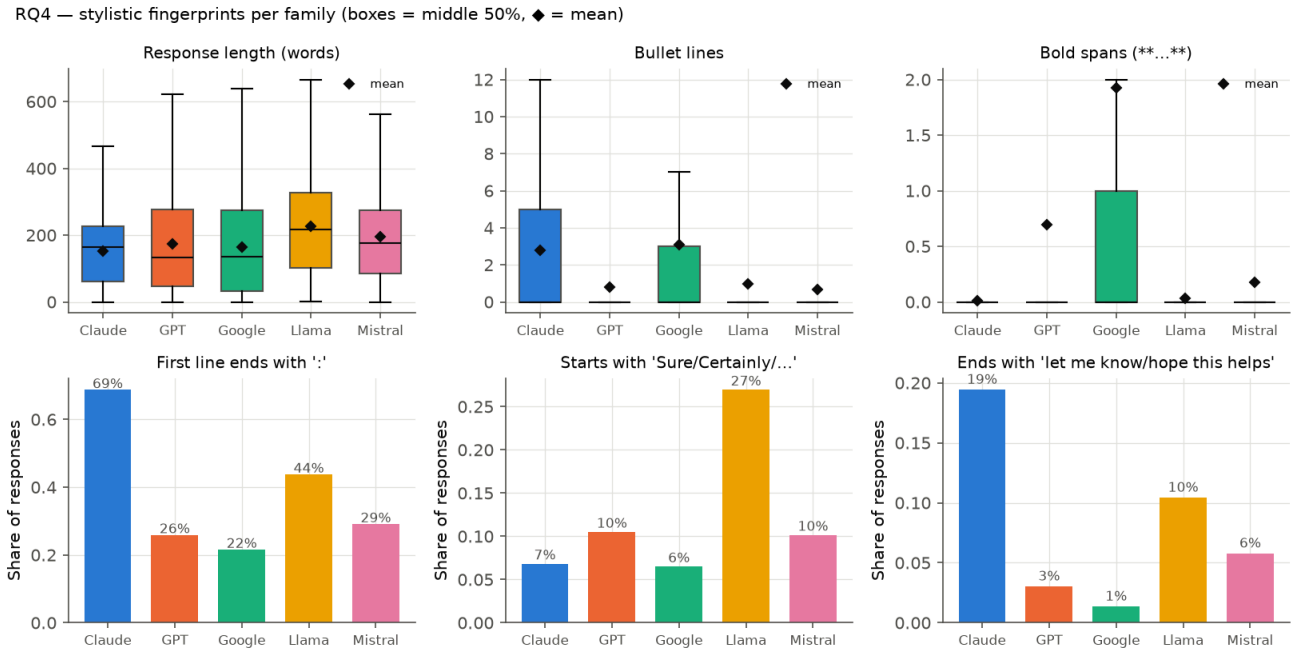


Figure 8. Selected style features per family. Lengths are capped for display.

- **Claude**: the shortest answers (153 words on average), the most paragraphs, many bullet lines, an introduction ending in a colon in 69% of responses ("Here is a basic explanation of …:"), and the most closing offers (19%).
- **GPT**: the highest lexical diversity (type–token and hapax ratios), apologies ("I'm sorry, but…") and the words *Certainly!*, *various*, *Remember,* and *Please note*.
- **Google** (Gemini/PaLM-2): by far the most **bold** text (1.9 bold spans per response vs 0.01 for Claude) and the most bullet points (3.1 per response).
- **Llama-2**: the longest answers (226 words), the lowest lexical diversity, opens with "Sure!" 27% of the time, the most numbered lists and emoji.
- **Mistral**: long answers, numbered lists, and the characteristic word "Note:"/"Note that" — otherwise close to GPT, which matches the confusions in Figure 2.

(b) How much can style alone do? A gradient-boosted tree classifier that sees **only the 29 numbers** — no words at all — reaches **0.59 macro-F1** (accuracy 0.60), i.e. 89% of the CNN's score. Trained on one feature group at a time, formatting is the strongest group (0.45), followed by lexical (0.36), punctuation (0.34), length (0.34) and phrasing (0.24). Permutation importance (how much F1 drops when one feature is shuffled) ranks: colon intro (0.078), number of paragraphs (0.058), bold spans (0.050), hapax ratio

(0.034), bullet lines (0.031), word count (0.028), numbered lines, sentence length, "Sure"-style openings and first-person rate.

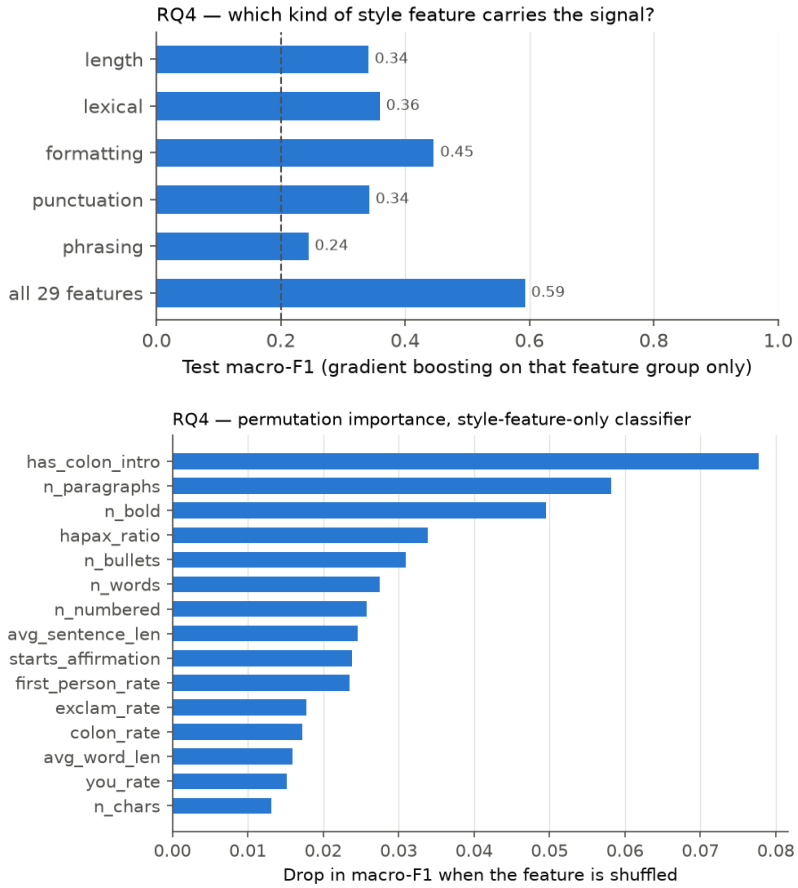


Figure 9. Top: F1 using one feature group at a time. Bottom: the 15 most important features.

(c) What words does a classifier use? The largest TF-IDF logistic-regression weights per family are:

Family	Most indicative n-grams (response text)
Claude	`Here is`, `Here are`, `•`, `: –`, `So`, `in summary`, `Okay ,`, `I do`
GPT	`Remember`, `Certainly !`, `As an`, `various`, `assist`, `m sorry`, `Please note`, `* :`
Google	`* *` (bold), `: *`, `. *`, `It is`, `be used`, `a number`
Llama	`Sure !`, `! Here`, `So ,`, `just an`, `m just` (an AI), `such as`, `: 1`
Mistral	`Note :`, `note that`, `\ *` (escaped markdown), `any questions`, `improve`

Table 5. The signals are mostly *function phrases and formatting*, not topic words.

(d) Ablations: remove one kind of signal and retrain. We transform every response (train, validation and test) and retrain the CNN (3 seeds) and the BiLSTM (1 seed, for time reasons):

- *No markdown / layout*: remove bullets, list numbers, headers, bold, code fences; collapse all whitespace.
- *Words only*: lower-cased words only (no punctuation, case, layout or emoji).
- *First 50 tokens*: keep only the beginning of the response (removes most length information).

- *Content words masked*: keep the 300 most frequent training tokens (function words, punctuation, markdown) and replace every other word by `<w>`; this removes topic while keeping style.
- *Word order shuffled*: randomly permute the tokens of each response (same bag of words, no order).

Training/test text	CNN (3 seeds)	Δ	BiLSTM (1 seed)	Δ
Original	0.647 ± .009	–	0.649	–
Content words masked	0.638 ± .001	−0.01	0.659	+0.01
No markdown / layout	0.571 ± .004	−0.08	0.586	−0.06
First 50 tokens only	0.558 ± .001	−0.09	0.579	−0.07
Words only (no punctuation/case/layout)	0.530 ± .016	−0.12	0.545	−0.10
Word order shuffled	0.501 ± .012	−0.15	0.542	−0.11

Table 6. Ablations (test macro-F1). Example of the masked text: "`<w> I do not have access to the number of <w> in my <w> . As an AI <w> without access to my own system <w> , I <w> <w> or <w> my own <w> .`"

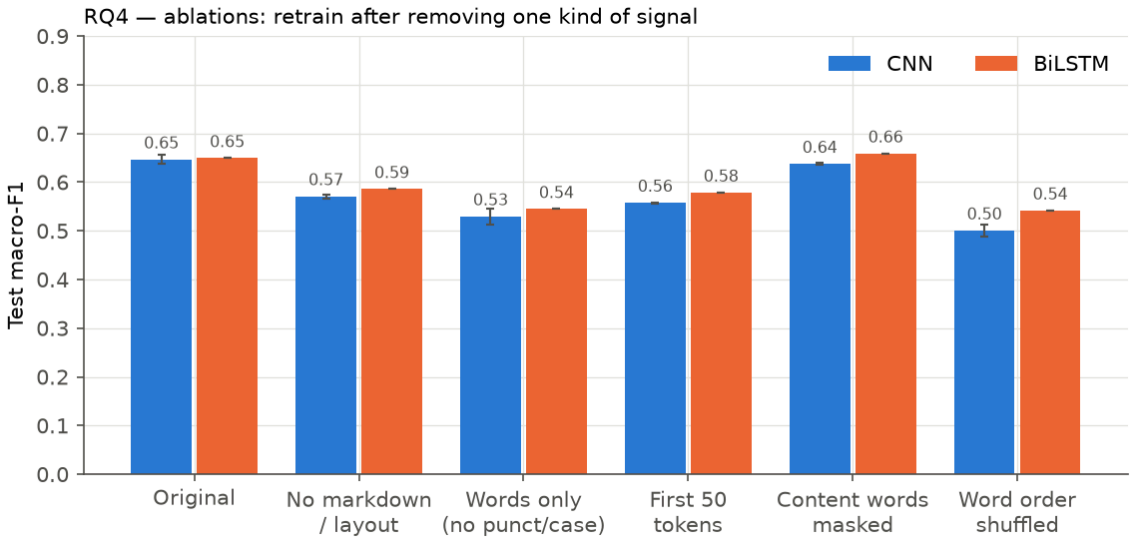


Figure 10. Ablations. Removing content words costs nothing; removing formatting, punctuation or word order costs a lot.

The results are very clear:

- **Topic is not the signal.** Replacing every word outside the 300 most frequent tokens by `<w>` leaves the scores unchanged (CNN −0.01, BiLSTM +0.01). The classifiers recognise *how* something is written, not *what* it is about, which is also why prompts (RQ2) do not help.
- **Formatting is worth ≈6–8 points.** Removing markdown and layout alone costs more than masking all content words.
- **Punctuation and case add another ≈4 points** on top of formatting (words-only vs no-markdown).
- **Length and the ending matter, but the opening carries most of the fingerprint.** With only the first 50 tokens the models keep ≈87–89% of their score — the opening sentence ("Here is …:", "Sure! Here's …", "Certainly!") is very distinctive.
- **Word order matters most.** Shuffling the tokens is the most damaging change (−0.15 for the CNN), so the signal is in *phrases* (n-grams), not in single words. The BiLSTM loses less than the CNN here

(-0.11), probably because its mean-pooled hidden states can still act like a bag-of-words summary, whereas the CNN's filters are built entirely on local word order.

Answers to the RQ4 questions. (i) The classifiers use formatting conventions (markdown, bullets, colons, paragraphing), fixed opening and closing phrases, and function-word habits much more than topic words. **(ii)** The signals are both structural and linguistic: formatting alone is the strongest single group, but no single group comes close to the full set, and the words-only model still works well above chance. **(iii)** Yes: removing formatting (-6 to -8 points), punctuation/case (a further -4), length (-7 to -9) or word order (-11 to -15) all reduce performance substantially, whereas removing topic words does not reduce it at all. LLM fingerprints are stylistic.

6. Lessons and experience

1. **How the data is split matters as much as the model.** Arena contains the same prompt many times. A random split by row would place copies of a prompt in both training and test sets. Splitting by prompt text, and checking that prompt-only accuracy is at chance, was our main guarantee that the reported numbers measure the models' style and not memorised prompts.
2. **A surprising input-only result means: check the data collection first.** RQ2 taught us to treat any "the prompt predicts the author" result as a warning sign of a sampling bias in how the data was collected.
3. **Always compare against a simple baseline.** Our TF-IDF + logistic regression model, trained in seconds, matched the BiLSTM. Without it we might have over-interpreted the neural results. Deep models are not automatically better on small datasets with no pre-training.
4. **Pre-processing choices are modelling choices.** Standard text cleaning (lower-casing, removing punctuation and newlines) would have deleted much of the signal we were looking for. Keeping newlines and markdown as tokens was essential.
5. **Tune on validation, report on test, use several seeds.** The first CNN looked much worse than it really was because its dropout was too strong and its training stopped too early. Seed-to-seed differences (± 1–2 points) are as large as some of the effects we measured, so single runs would have been misleading.
6. **Engineering problems take real time.** Packed LSTMs were $8\times$ slower on the Mac GPU, and the GPU memory cache grew with every new batch shape until the machine started swapping (CNN epochs went from 7 s to 150–580 s). Writing our own bidirectional LSTM (and verifying it numerically against PyTorch's) and rounding padded lengths to a multiple of 32 fixed both problems.
7. **Accuracy tells you *whether*, analysis tells you *why*.** The ablations and the interpretable features turned a single number (66%) into concrete knowledge: Claude can be recognised by its colon introductions and bullets, Google by bold text, Llama by "Sure!", Mistral by "Note:". These cues are also easy to change with a system prompt or a post-processing step, so a real-world LLM detector built on them would be easy to fool.

7. Limitations and future work

- Only 2023 – early-2024 models; newer versions (GPT-4o, Claude 3+, Llama 3, Gemini 1.5+) may behave differently.

- Family labels merge versions of very different size; per-version classification would be harder and is left for future work.
- Task labels come from a keyword heuristic and contain some mistakes.
- The models learn embeddings from scratch; fine-tuning a pre-trained transformer (e.g. RoBERTa) would likely score higher and could be combined with the style features.

8. Reproducibility and use of AI tools

All code, the processed dataset, per-run results (JSON), training logs and figures are in the repository; `run_all.sh` reproduces every number in this report. As allowed by the course policy, we used an AI assistant (Claude) to help write and debug code, plan experiments and draft text. We ran all experiments ourselves and checked the results; the team is responsible for and able to explain all of the submitted work.